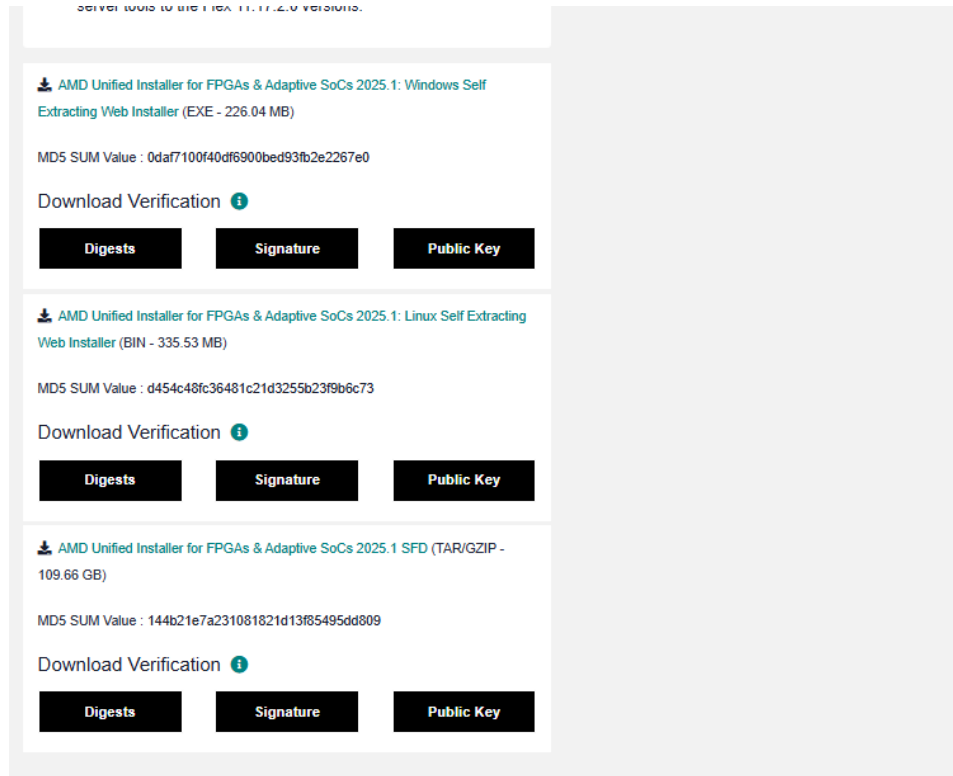


At the time of writing, two versions of Vivado are available:

- Vivado™ Edition 2025.1 – Full Product Installation This is the complete installation of the Vivado Design Suite. It includes all tools needed for design, simulation, synthesis, implementation, and device programming.
- Vivado Lab Solutions 2025.1 This is a lightweight version intended mainly for programming and debugging FPGA devices in a lab environment. It does not include the full design and synthesis tools.

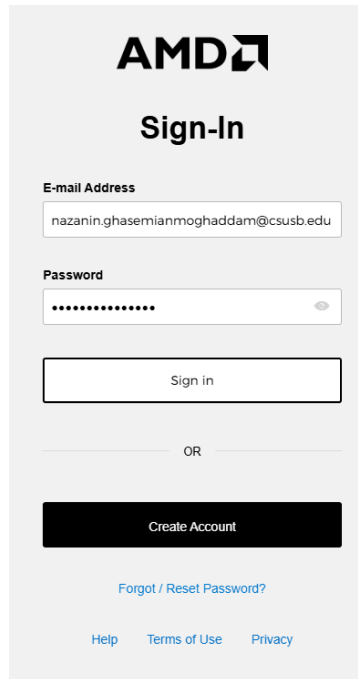
For this course, we will use the first option: Vivado™ Edition 2025.1 – Full Product Installation.



1.1 Download the Vivado Design Suite

Select the appropriate installer for your operating system (Windows or Linux). Note that there is no official version of Vivado for macOS. If you are using a Mac, you will need to run Vivado inside a virtual machine (VM) with a supported operating system.

You will need an AMD account to download Vivado. Register using your CSUSB email address, then sign in to access the download. The Windows installer is approximately 230 MB in size. This installer will both download the required files and guide you through the installation process.

The image shows the AMD Sign-In page. At the top is the AMD logo. Below it is the heading "Sign-In". There are two input fields: "E-mail Address" with the text "nazanin.ghasemianmoghaddam@csusb.edu" and "Password" with masked characters. Below the password field is a "Sign in" button. Underneath is a horizontal line with "OR" in the center. Below that is a black "Create Account" button. At the bottom, there is a link "Forgot / Reset Password?" and three links: "Help", "Terms of Use", and "Privacy".

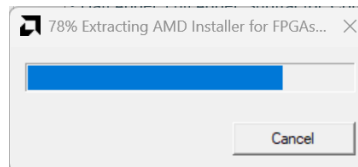
1.2 Install the Vivado Design Suite

This section explains the installation process of the Vivado Design Suite for all supported platforms.

Before extracting and installing the tool, keep the following points in mind:

- Ensure that your machine meets the system requirements for this toolchain. You can verify this on the AMD website.
- Confirm that you have sufficient disk space available. The installation may require several gigabytes of storage.

Now, start the installer you downloaded in the previous step. When you launch the installer, it will automatically extract the necessary files and prepare the system for installation.



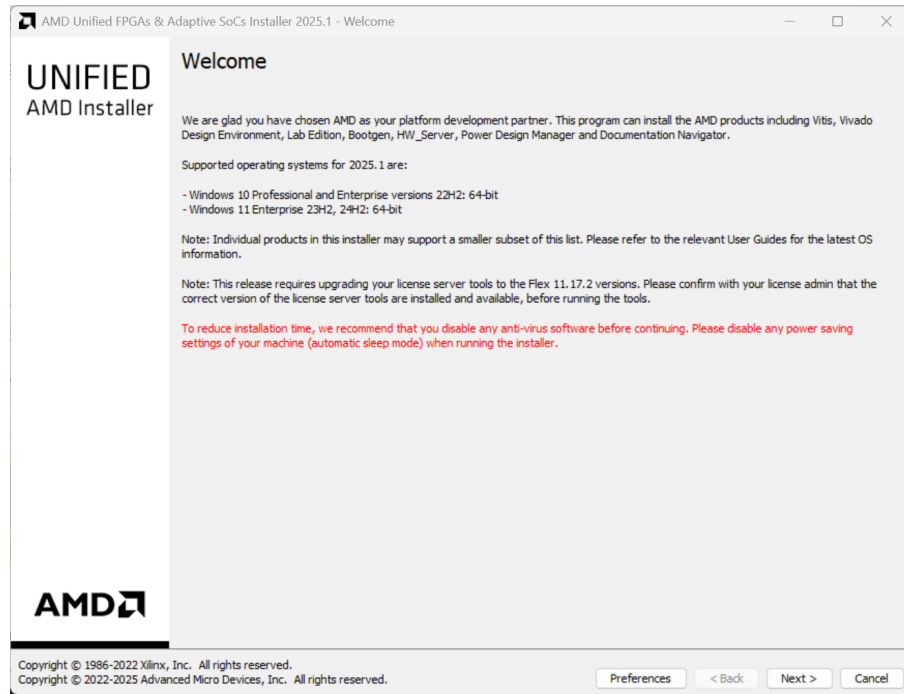
To begin the installation process on Linux:

- Open a terminal and run:

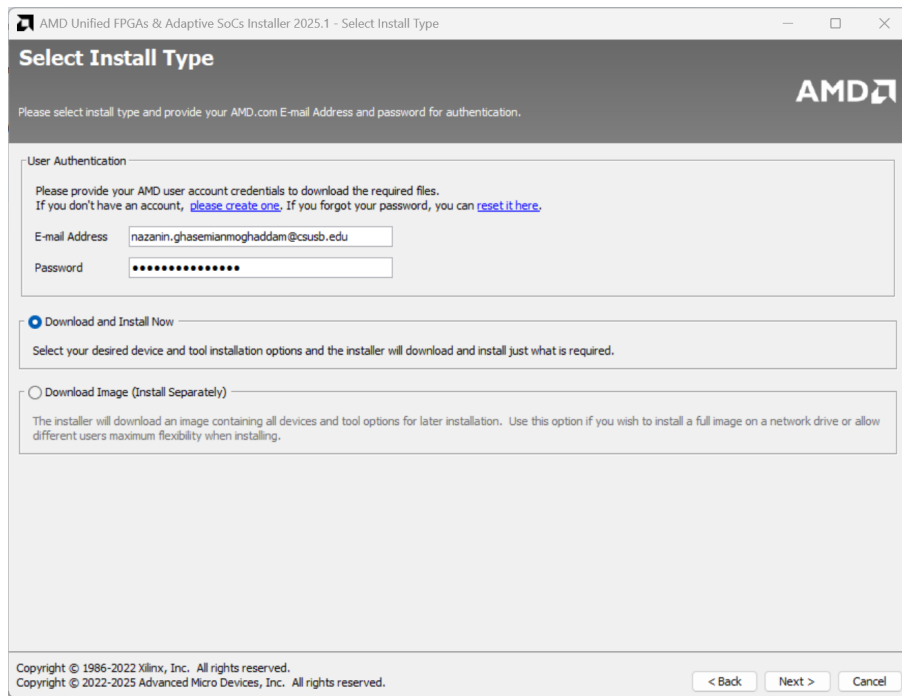
```
$chmod +x FPGAs_AdaptiveSoCs_Unified_SDI_2025.1_0530_0145_Lin64.bin  
  
$sudo ./FPGAs_AdaptiveSoCs_Unified_SDI_2025.1_0530_0145_Lin64.bin
```

- The Vivado Installer window will appear. The installation process is the same as on Windows.

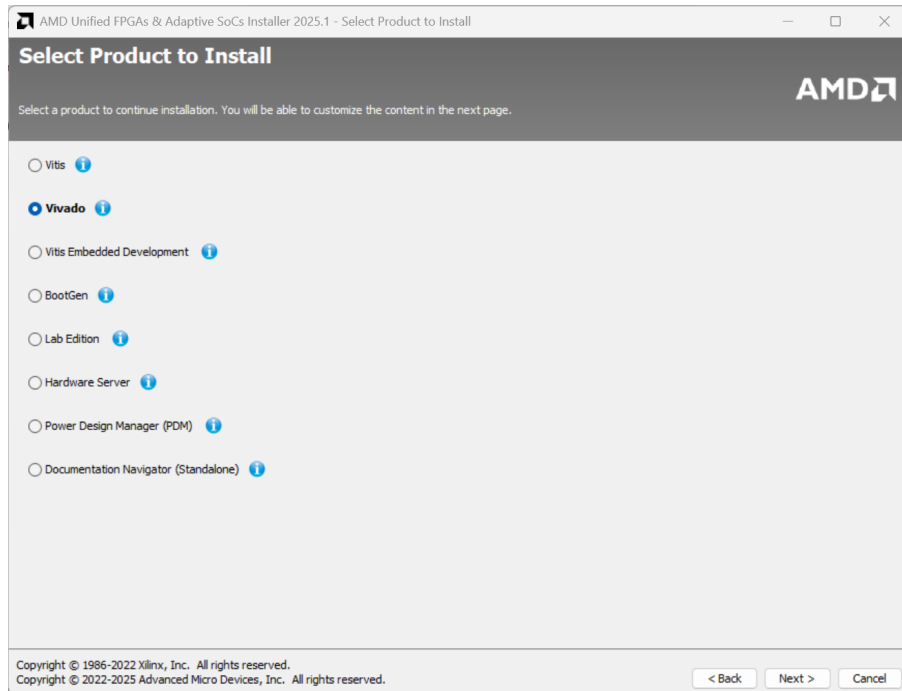
This is the first window that appears in the installation process. Click Next to continue.



A window will appear asking for your installation type and AMD account. Sign in with the account you created earlier on the AMD website. Select Download and Install Now, then click Next to continue.



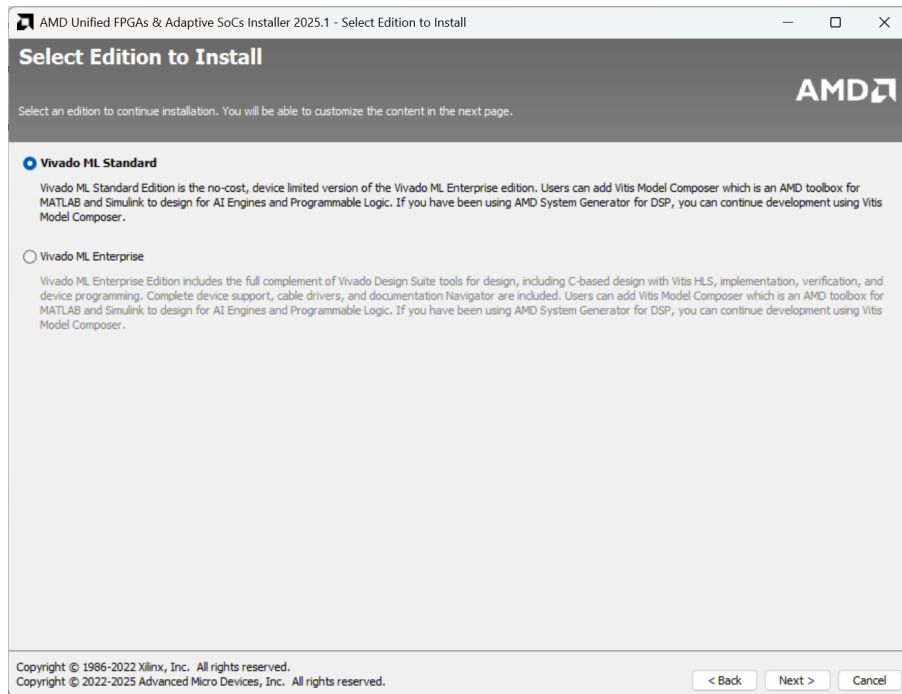
This window asks you to choose which tool to install. Select Vivado and click Next to continue.



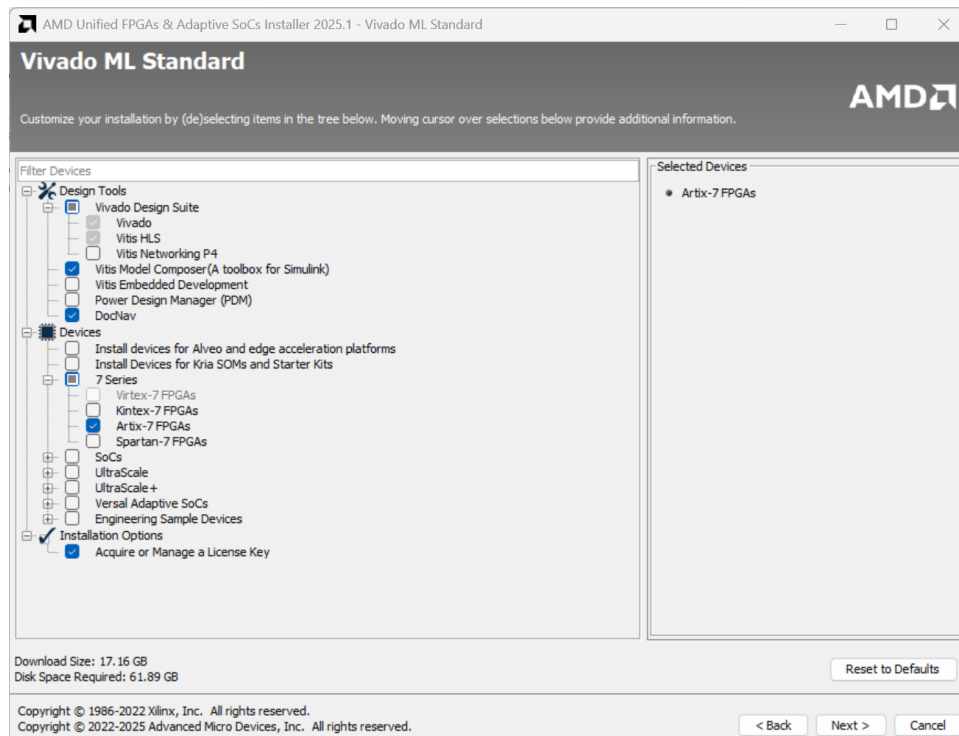
1.3 License and Edition Selection

In the next step, you will select which edition of the Vivado tool to install. For our purposes, the Standard edition is sufficient and does not require a license.

Please select "Vivado ML Standard".



In this step, select which parts of the software to include in the installation. The default options are usually sufficient, but at least one device family must be selected. Here, we chose the Artix-7 family. Other device families can be downloaded later if needed. Once finished, click Next.



Review and accept the license agreement by selecting all “I Agree” checkboxes. Ensure you scroll through the entire agreement and confirm all boxes are checked.

AMD Unified FPGAs & Adaptive SoCs Installer 2025.1 - Accept License Agreements

Accept License Agreements

Please read the following terms and conditions and indicate that you agree by checking the I Agree checkboxes.

☒ I Agree

End User License Agreement for Model Composer

By checking "I AGREE" below, or OTHERWISE ACCESSING, DOWNLOADING, INSTALLING or USING THE SOFTWARE, I AGREE on behalf of licensee to be bound by the agreement, which can be viewed by [clicking here](#).

☒ I Agree

Third Party Software End User License Agreement for DocNav

By checking "I AGREE" below, or OTHERWISE ACCESSING, DOWNLOADING, INSTALLING or USING THE SOFTWARE, YOU AGREE on behalf of licensee to be bound by the agreement, which can be viewed by [clicking here](#).

☒ I Agree

Third Party Software End User License Agreement for Vivado

By checking "I AGREE" below, or OTHERWISE ACCESSING, DOWNLOADING, INSTALLING or USING THE SOFTWARE, YOU AGREE on behalf of licensee to be bound by the agreement, which can be viewed by [clicking here](#).

☒ I Agree

Third Party Software End User License Agreement for Model Composer

By checking "I AGREE" below, or OTHERWISE ACCESSING, DOWNLOADING, INSTALLING or USING THE SOFTWARE, YOU AGREE on behalf of licensee to be bound by the agreement, which can be viewed by [clicking here](#).

☒ I Agree

Copyright © 1986-2022 Xilinx, Inc. All rights reserved.
Copyright © 2022-2025 Advanced Micro Devices, Inc. All rights reserved.

< Back Next > Cancel

Select the desired installation directory, then proceed by clicking Next.

AMD Unified FPGAs & Adaptive SoCs Installer 2025.1 - Select Destination Directory

Select Destination Directory

Choose installation options such as location and shortcuts.

Installation Options

Select the installation directory

C:\Xilinx

Installation location(s)

- C:\Xilinx\2025.1\Vivado
- C:\Xilinx\2025.1\Wits
- C:\Xilinx\2025.1\Model_Composer
- C:\Xilinx\DocNav

Download location

C:\Xilinx\Downloads\Vivado_2025.1

Disk Space Required

Download Size:	17.16 GB
Disk Space Required:	62.59 GB
Final Disk Usage:	45.43 GB
Disk Space Available:	84.53 GB

Select shortcut and file association options

☒ Create program group entries

Xilinx Design Tools

☒ Create desktop shortcuts

☒ Create file associations

Apply shortcut & file association selections to

☒ Current user

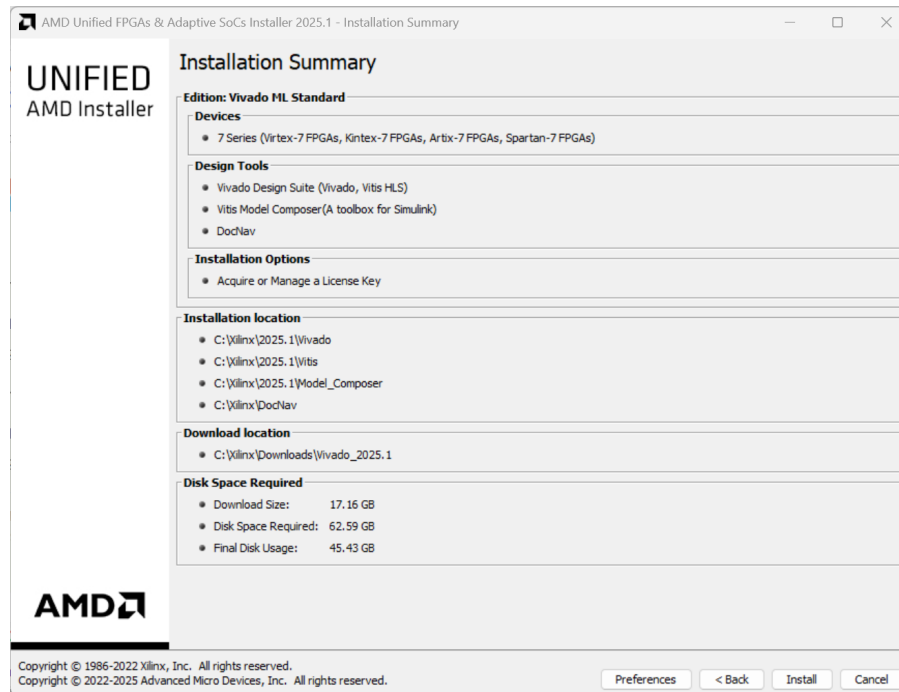
☐ All users

If you are installing multiple products in the same directory, please note that during the uninstallation process all products will be removed from this directory.

Copyright © 1986-2022 Xilinx, Inc. All rights reserved.
Copyright © 2022-2025 Advanced Micro Devices, Inc. All rights reserved.

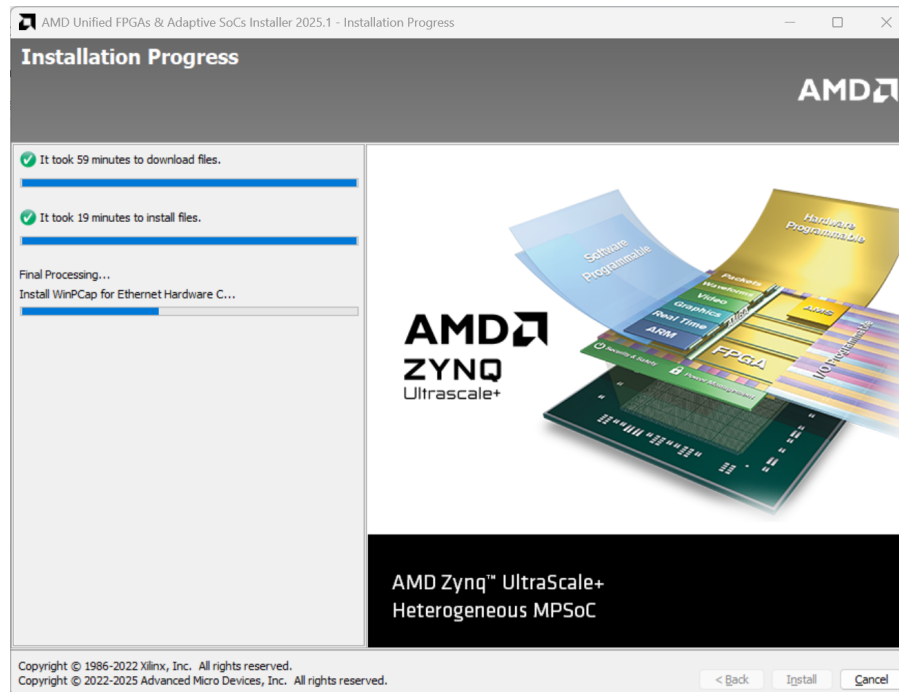
< Back Next > Cancel

Start the installation by clicking Install.



1.4 Download and installation

In the final stage of installation, the full image of the Vivado tool will be downloaded. Even with a fast internet connection, this step can take a considerable amount of time, so it is recommended to complete it before class. Once the download is finished, the installation process will begin automatically.

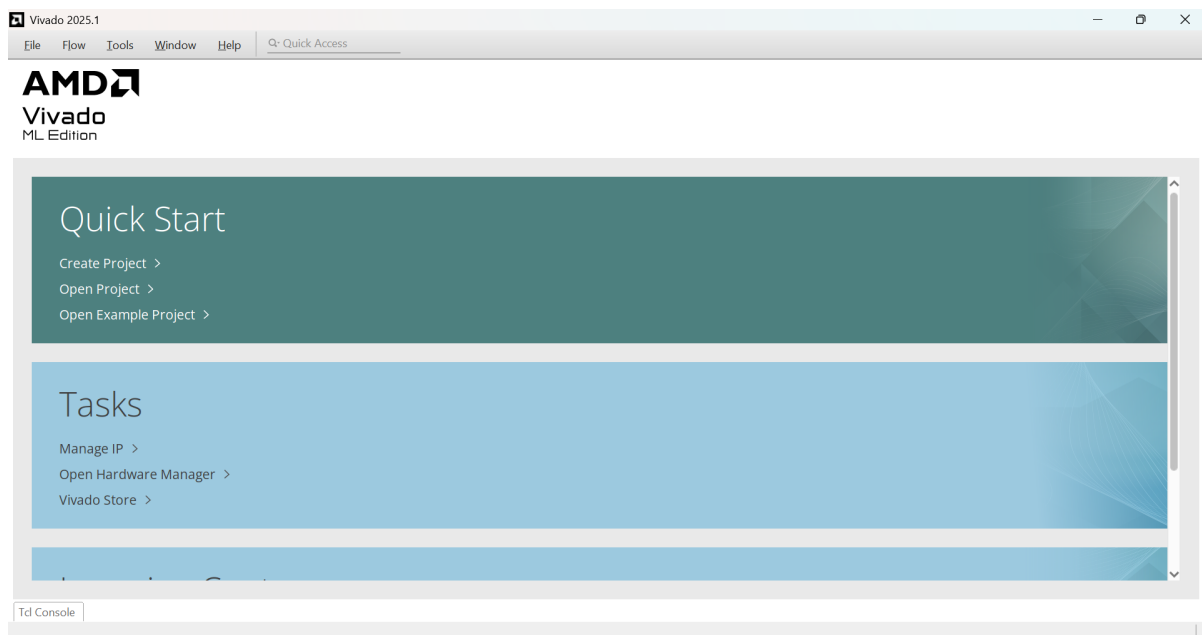


2 Using The Vivado Design Suite

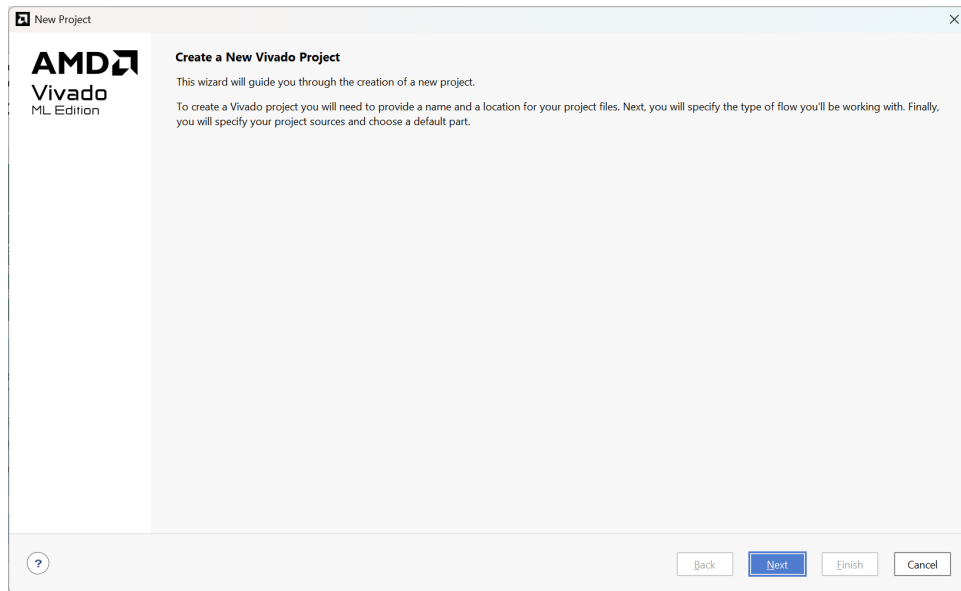
Open Vivado from your desktop.



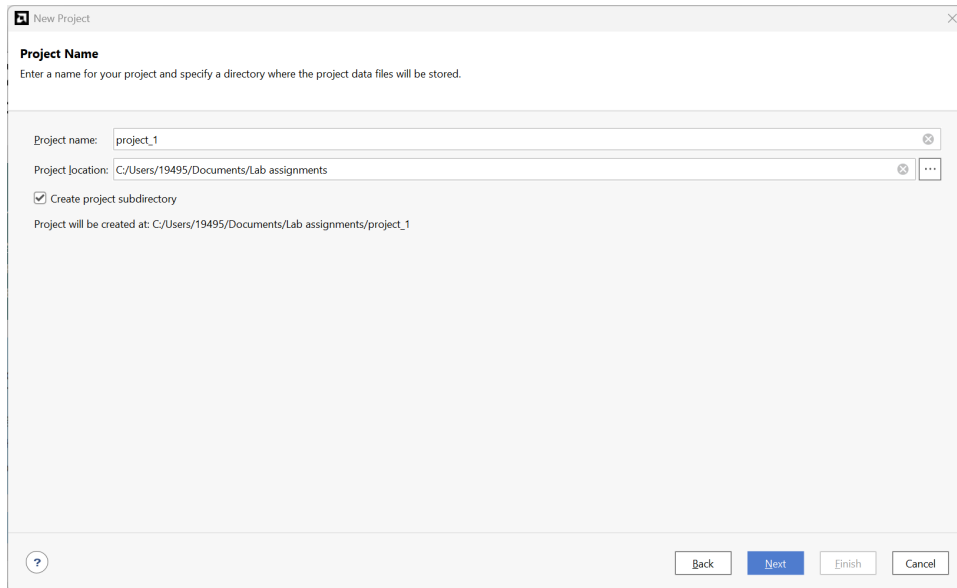
Select “Create Project”.



Click next.



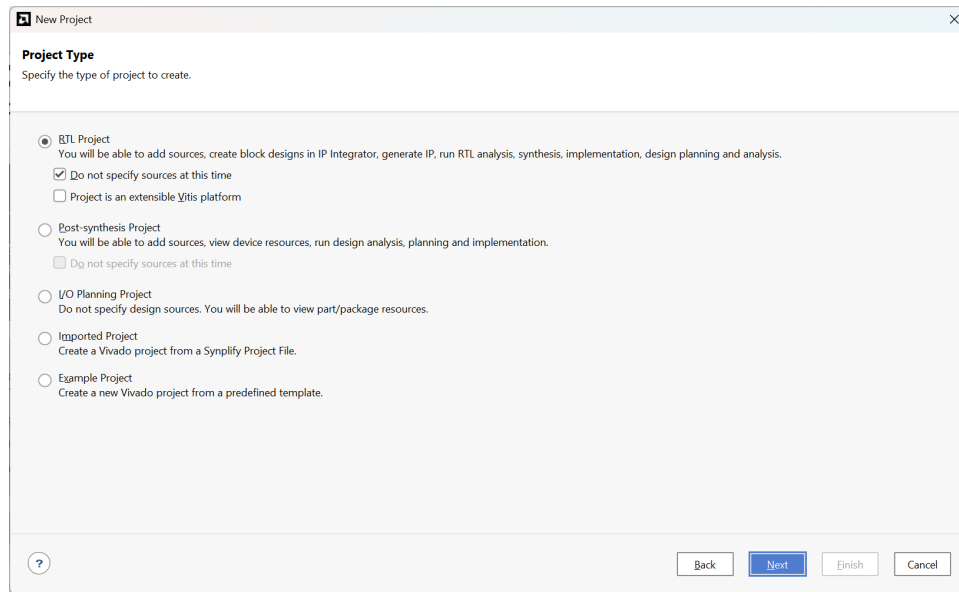
When creating a new project, you must provide a name and a location on your computer. This window prompts you to enter a project name and specify the directory where the project should be saved.



Vivado Naming Conventions:

Project names and source file names must start with a letter ($A - Z, a - z$) and must contain only alphanumeric characters ($A - Z, a - z, 0 - 9$) and underscores($_$).

This window asks you to specify the type of project. In the Project Type section, select RTL Project. Then check the option Do not specify sources at this time (we will discuss this later). Click Next to continue.



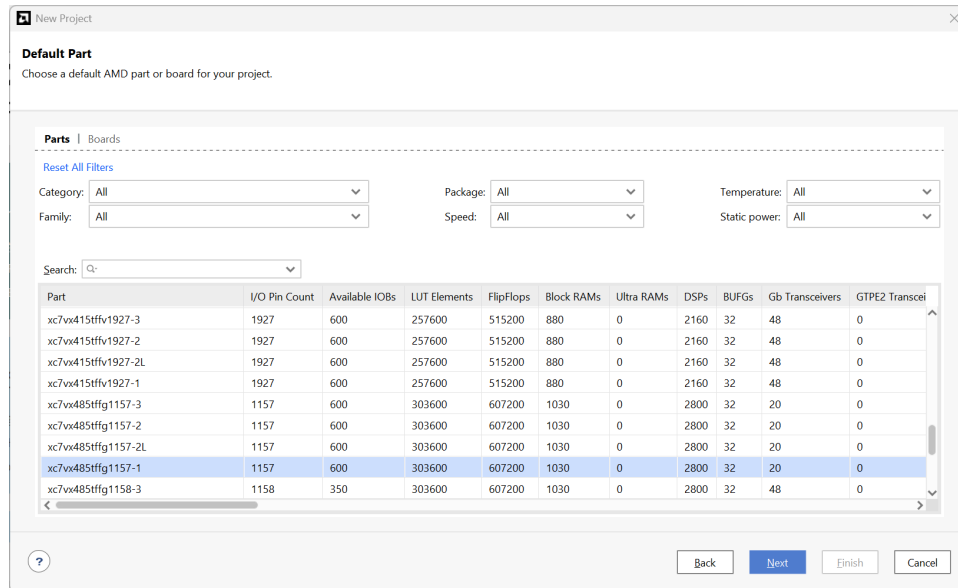
Note:

Vivado allows you to create different types of projects. The options are:

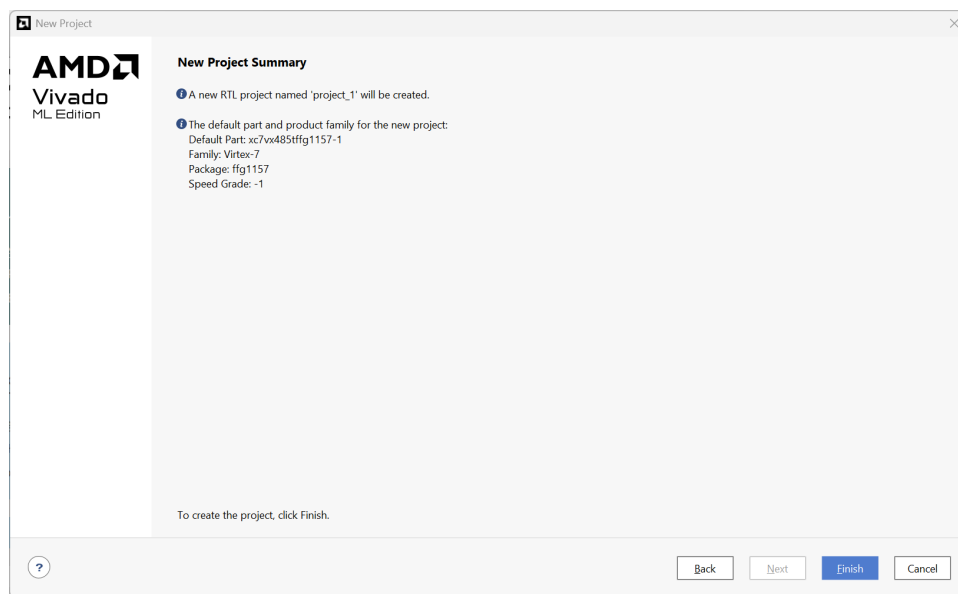
- **RTL Project**
Create a design starting from HDL sources or IP. This type of project supports the full design flow, from RTL analysis through synthesis, implementation, and hardware validation. You can also:
 - Import existing RTL and IP to build your design.
 - Create standalone reusable IP modules.
 - Start with an empty project to explore device resources.
- **Post-synthesis Project**
Begin with a synthesized netlist (e.g., from a third-party tool) and continue through implementation, analysis, and hardware validation.
- **I/O Planning Project**
Focus on early I/O pin assignments without adding design sources. You can define ports manually or import them from files (CSV, RTL, or XDC). These projects can later be migrated into an RTL Project.
- **Imported Project**
Import projects from tools such as Synplify, XST, or ISE. Source files and their compilation order are migrated, but synthesis and implementation results or tool settings are not.
- **Example project**
Start from a predefined Vivado example project, useful for learning and demonstrations.

In the Default Part dialog box, you would normally select the FPGA family and part number on which

your design will be implemented. However, for now, you can skip this step and simply click Next.



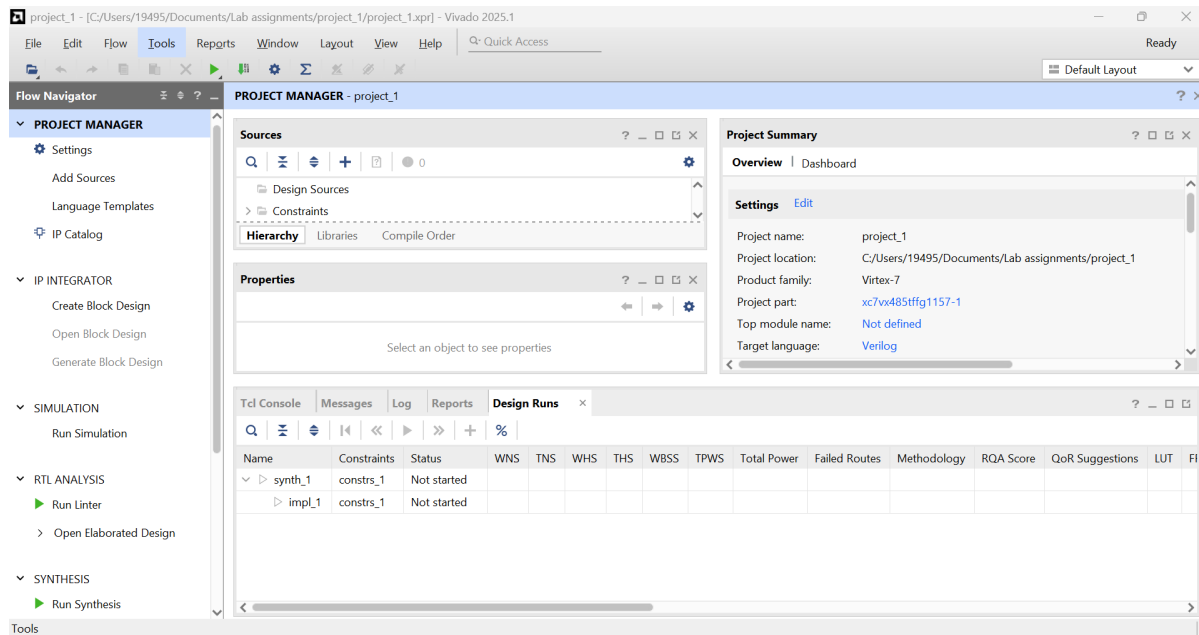
In the New Project Summary dialog box, review the project settings carefully. Once everything looks correct, click Finish to create the project.



When the project opens, you will see several main windows in Vivado, such as:

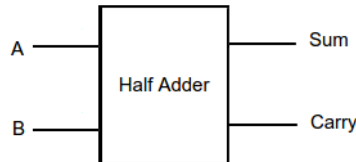
- **Flow Navigator** – access to design steps like simulation, synthesis, and implementation.
- **Sources** – a list of your design and constraint files.
- **Project Summary** – key information about the project and its current status.

These windows form the basic workspace you'll use throughout the course.



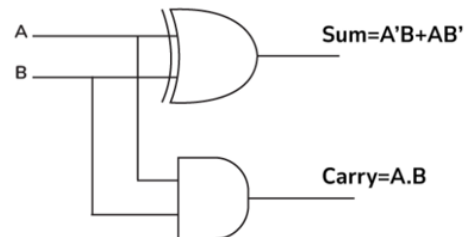
2.1 Design a Half Adder

We want to design a Half Adder. A Half Adder (HA) is a basic combinational circuit that adds two single-bit binary numbers. It produces two outputs: the Sum and the Carry. The block diagram is shown below.



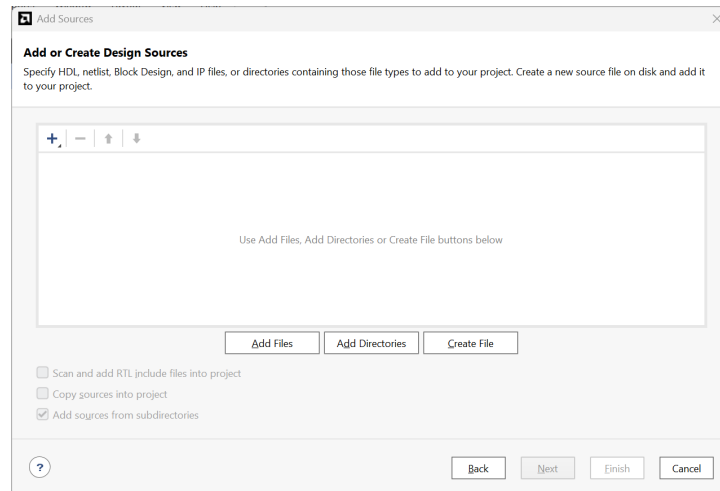
Here is the truth table of the Half Adder. As you can see, the Sum output is the XOR of the two inputs, while the Carry output is the AND of the two inputs.

Truth Table			
A	B	S	C
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

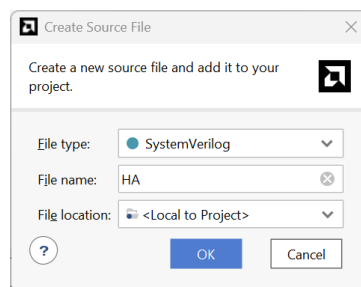


Now we will implement the Half Adder in Vivado. In the Flow Navigator window, select Add Sources. Choose Add or Create Design Sources, then click Next.

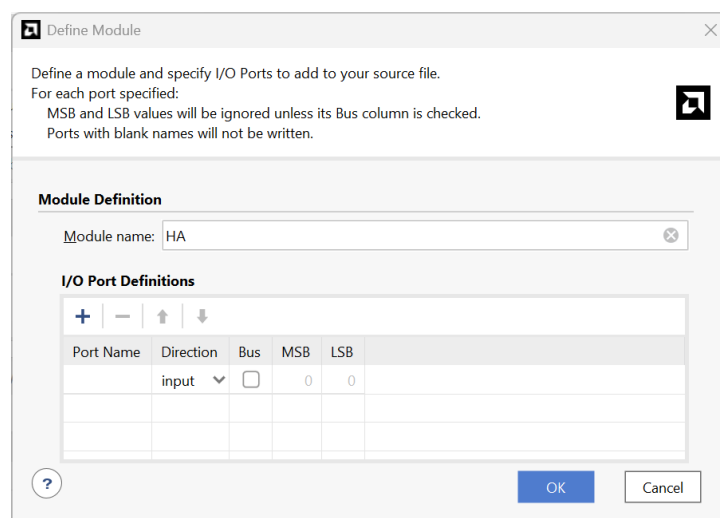
We want to create a new file so, click "Create File".



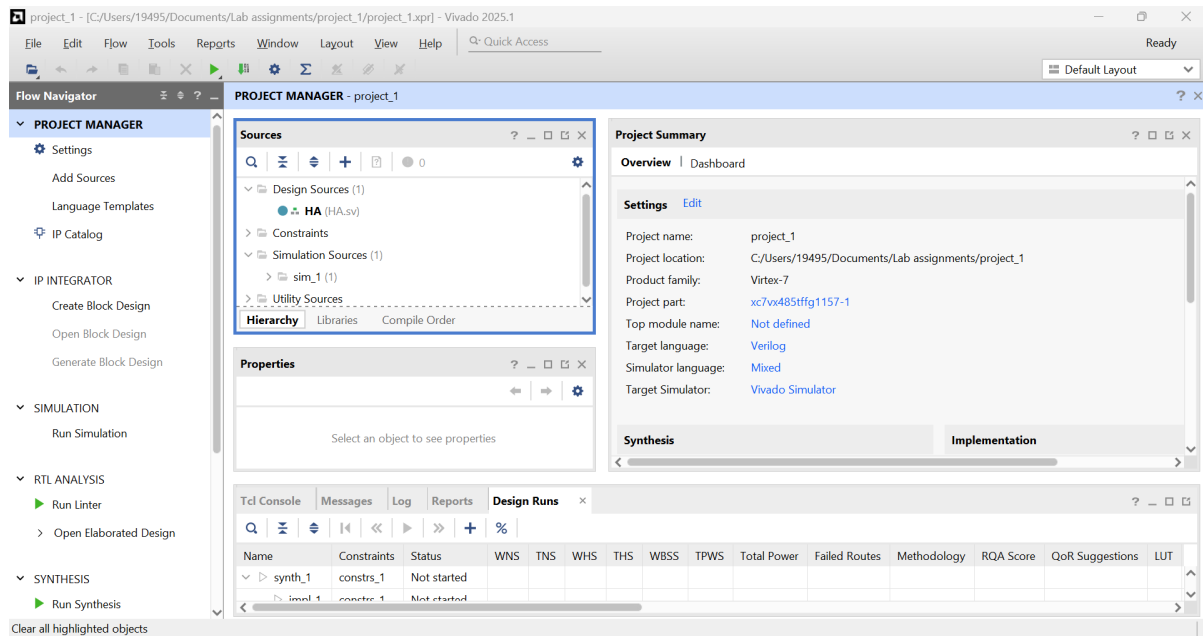
Define a name for your design, change the file type to SystemVerilog, then click OK and Finish.



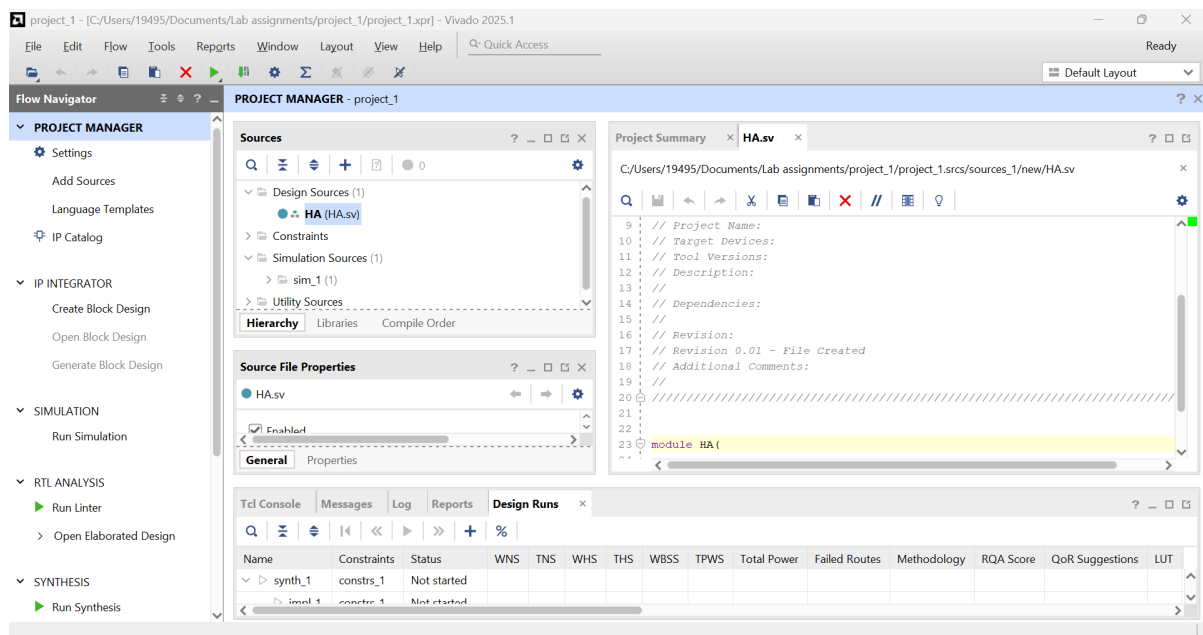
The next window asks you to define I/O ports for your module. Since we will define them later in the code, skip this step and click OK.



You should now see your new source file (HA) listed under Design Sources in the Sources window.



Double-click on HA to open the file.



Below is the SystemVerilog code for a Half Adder. Copy and paste this code into the HA.sv window.

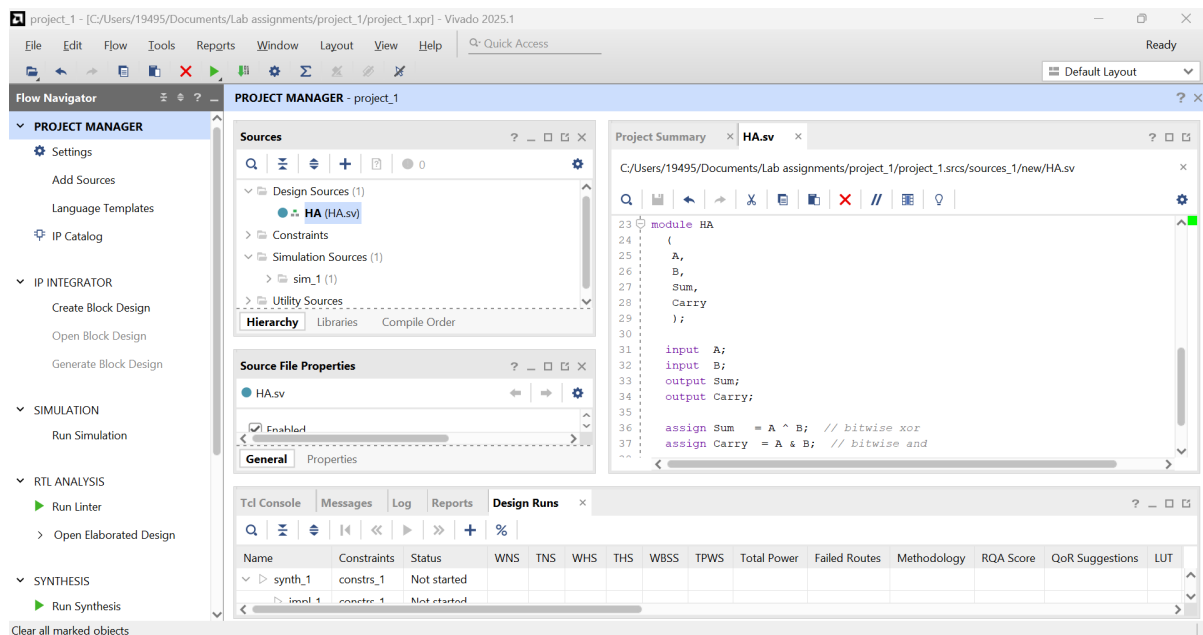
Code 1: Half Adder.

```
module HA
(
    A,
    B,
    Sum,
    Carry
);

input A;
input B;
output Sum;
output Carry;

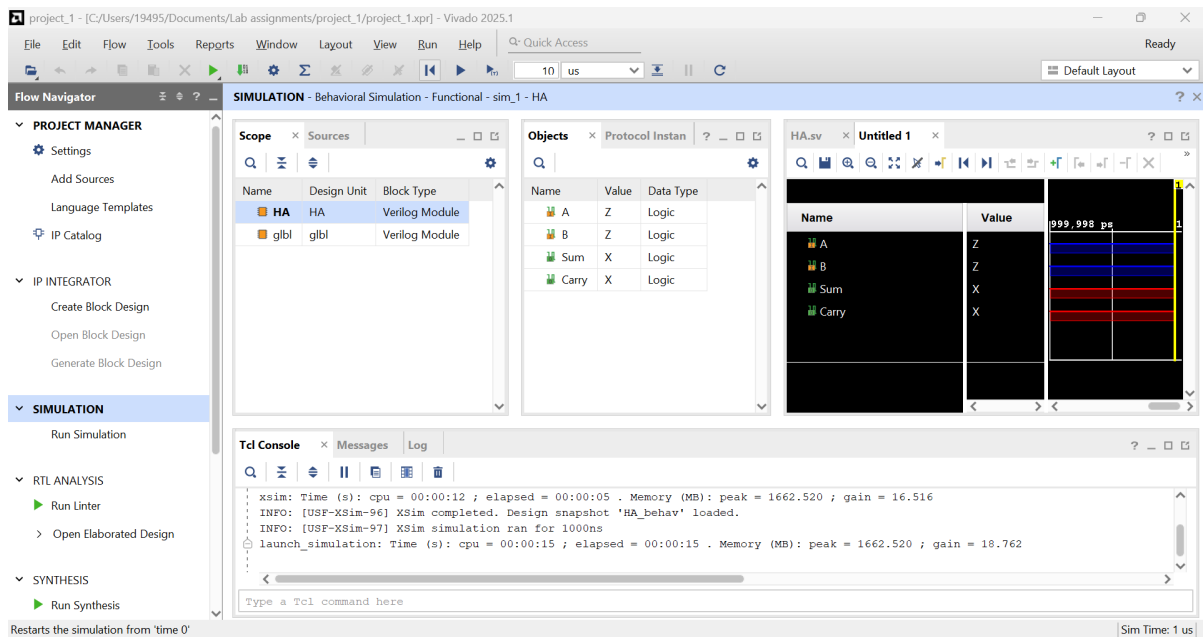
assign Sum = A ^ B; // bitwise xor
assign Carry = A & B; // bitwise and
endmodule // half_adder
```

If there is a syntax error, the line containing the error will be underlined in red. Be sure to correct all errors, then save your code.



2.2 Simulation

To simulate your design, click Run Simulation in the Flow Navigator window and select Run Behavioral Simulation. If the simulation runs successfully, the simulation window will open.

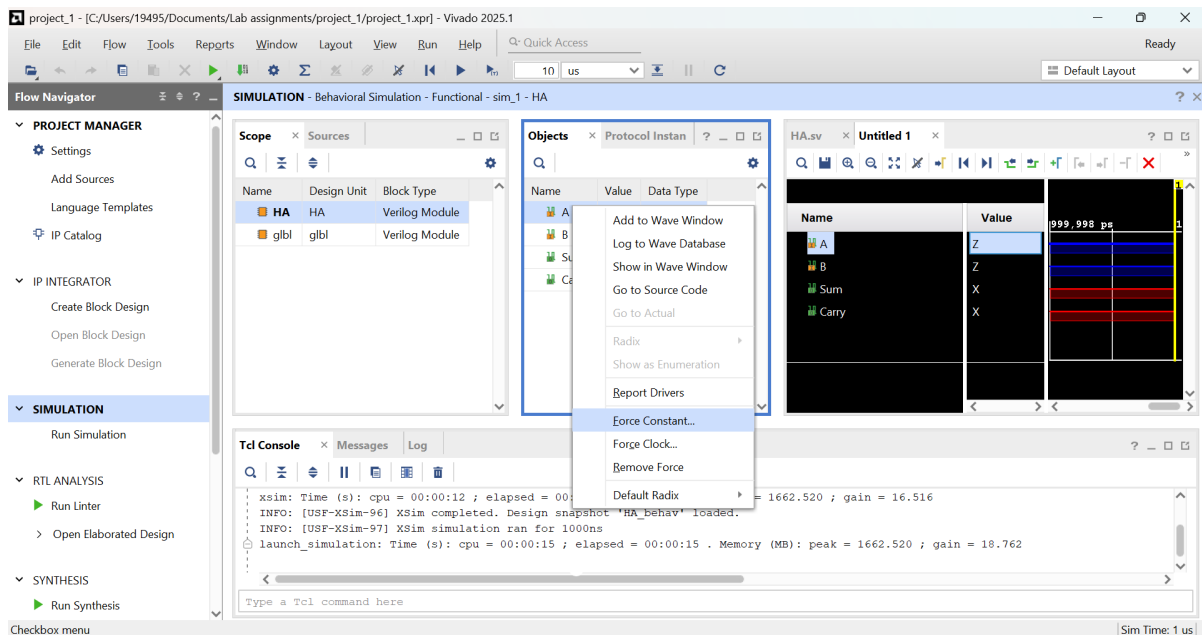


You can verify the functionality of your design using the simulation window. If the results are not as expected, return to your design code, make the necessary corrections, and re-run the simulation.

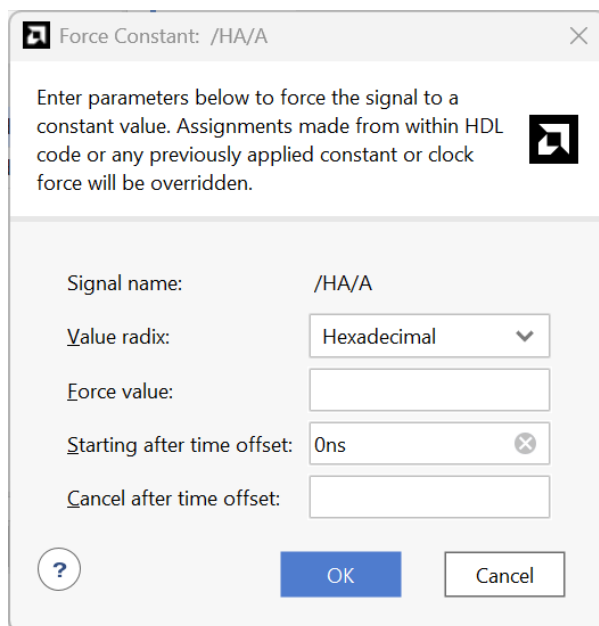
To test the Half Adder, we need to provide values for its two inputs, A and B. The table below shows two sample test scenarios. You can extend this by trying all possible input combinations (00, 01, 10, 11) to fully confirm the design's correctness.

Inputs		Outputs	
A	B	Sum	Carry
1	0	1	0
1	1	0	1

In the Objects window right click on A and select "Force Constant".



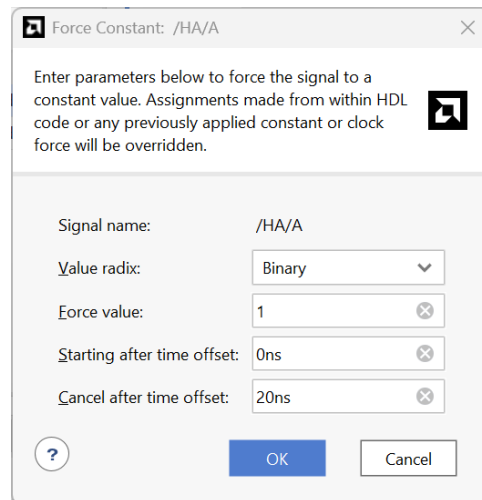
The Force Constant window has several fields:



- Signal name: Shows the default signal name, which is the full path of the selected item.
- Value radix: Displays the current radix (number format) for the selected signal. You can choose from Binary, Hexadecimal, Unsigned Decimal, Signed Decimal, Octal, or ASCII. Based on your selection, only valid values can be entered. For example, if you select Binary, you can only enter 0 or 1.
- Force value: Allows you to assign (or “force”) a constant value to the signal using the selected radix.

- Starting at time offset: Defines the simulation time when the forced value will begin. The default is 0. You can specify a plain number (e.g., 10) or include units (e.g., 10 ns). If no unit is given, Vivado uses the default simulation unit.
- Cancel after time offset: Defines the time at which the forced value will be removed. Similar to the starting offset, you can specify a number (e.g., 10) or a time with units (e.g., 10 ns).

For our example, input **A** is a 1-bit signal, so set the Value radix to Binary. In the first test, we want $A = 1$. Set the Force value to 1, with a Starting time of 0 ns and a Cancel time of 20 ns. Once these are entered, click OK to apply.



Force Constant: /HA/A

Enter parameters below to force the signal to a constant value. Assignments made from within HDL code or any previously applied constant or clock force will be overridden.

Signal name: /HA/A

Value radix: Binary

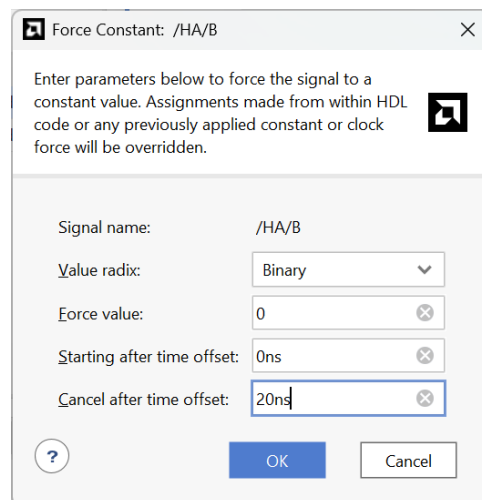
Force value: 1

Starting after time offset: 0ns

Cancel after time offset: 20ns

OK Cancel

Do the same thing for B. Note: in the first test B is 0.



Force Constant: /HA/B

Enter parameters below to force the signal to a constant value. Assignments made from within HDL code or any previously applied constant or clock force will be overridden.

Signal name: /HA/B

Value radix: Binary

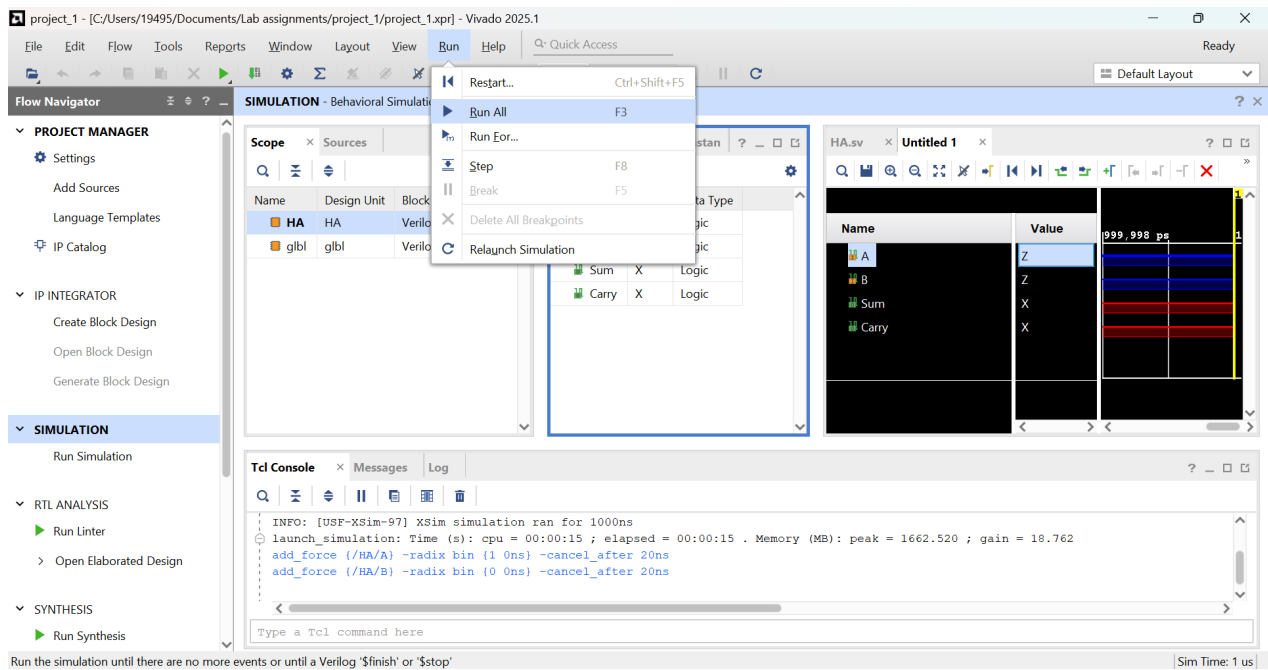
Force value: 0

Starting after time offset: 0ns

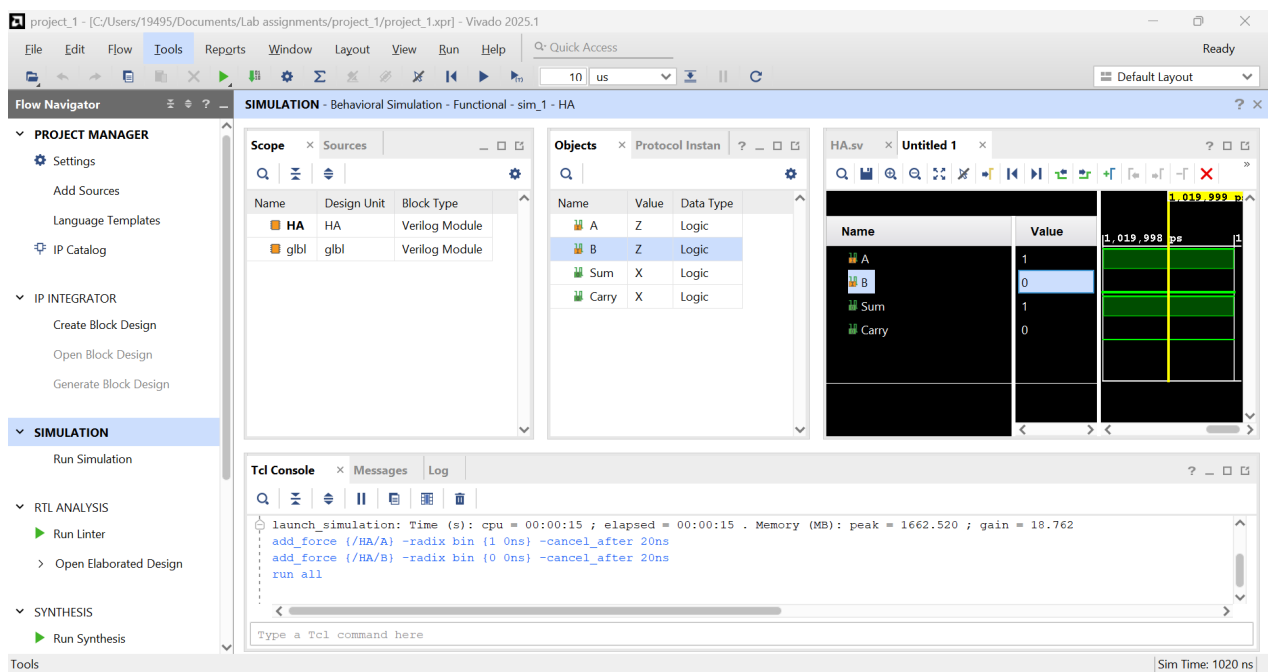
Cancel after time offset: 20ns

OK Cancel

Now from the top menu select Run All.

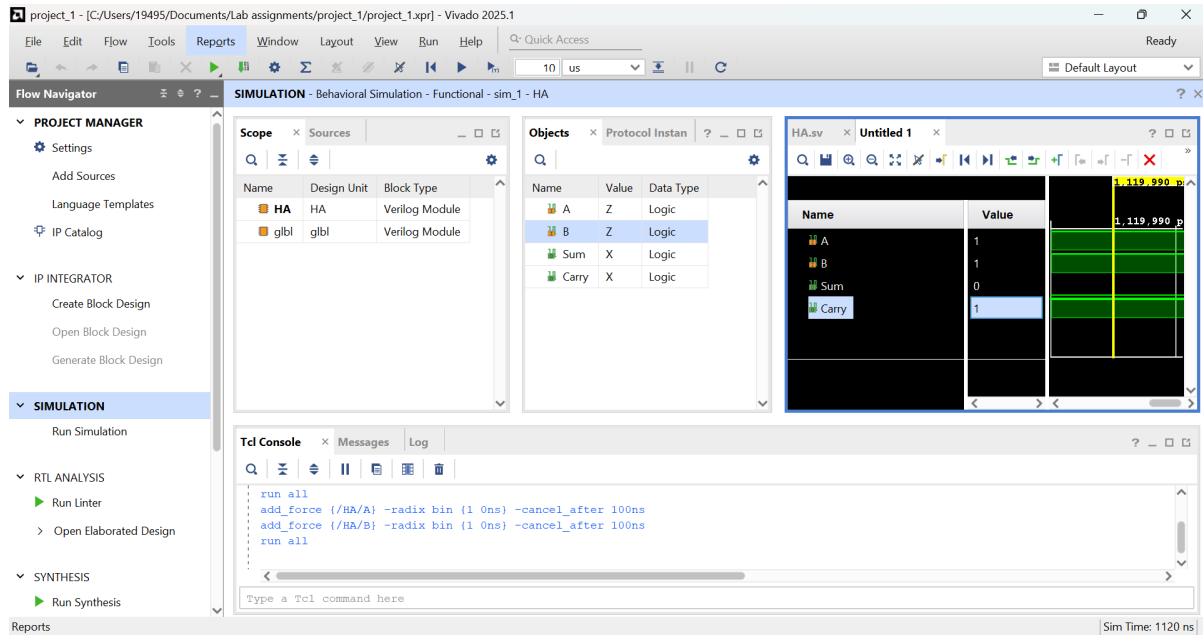


You can view the simulation results in the simulation window. Move the yellow cursor line across the waveform diagram to examine different points in time. Notice that when input A is set to 1 and B is 0, the outputs are Sum = 1 and Carry = 0, which is the expected result.



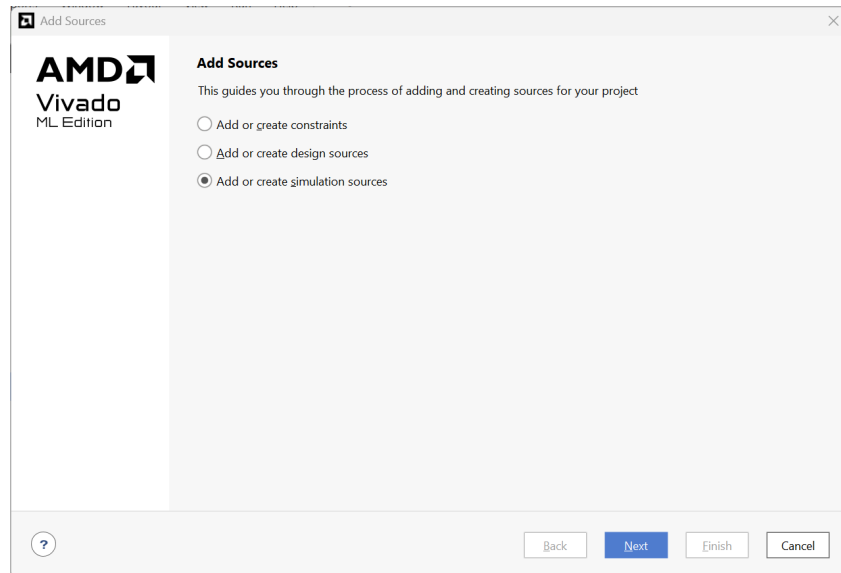
Now, run the second test by setting both inputs A and B to 1. Observe the simulation results: the Sum

output is 0 and the Carry output is 1. This matches the expected behavior of a Half Adder, since adding $1 + 1$ in binary produces a result of 10 (Sum = 0, Carry = 1).



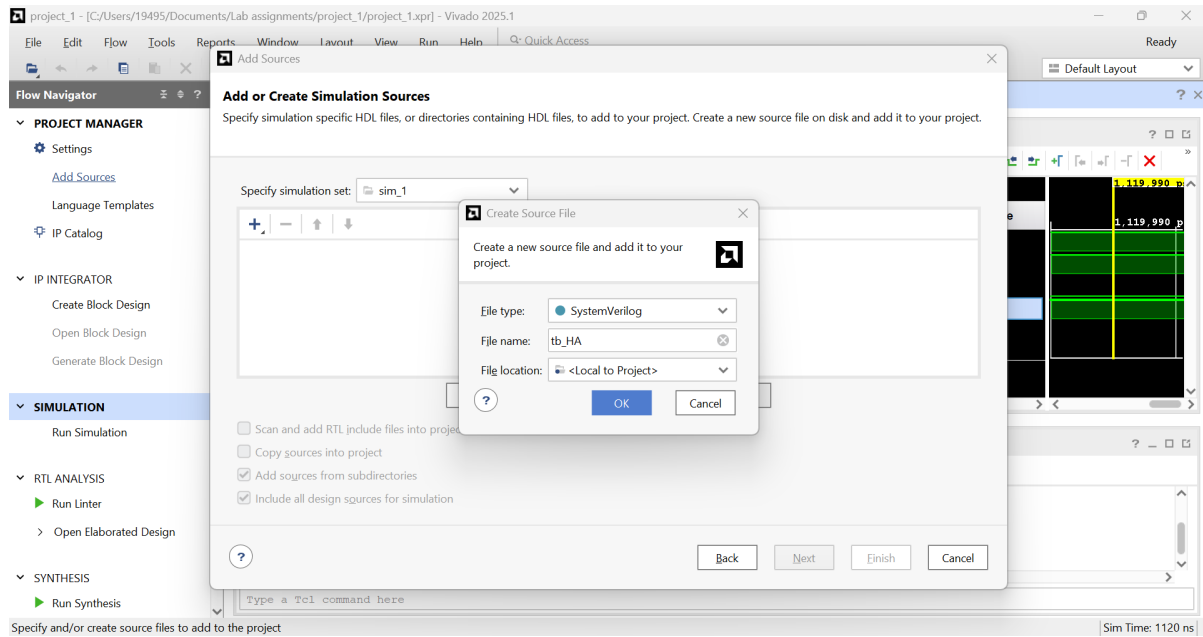
Another way to test your design is by writing a testbench code. A testbench is a separate piece of code that applies different input values to your design automatically and checks the outputs, rather than forcing signals manually in the simulation window. This makes testing faster and more reliable, especially for larger designs.

To create a testbench in Vivado, go to the Flow Navigator window and select Add Sources. This time, choose Add or Create Simulation Sources. This will allow you to add a new SystemVerilog file that will serve as your testbench.

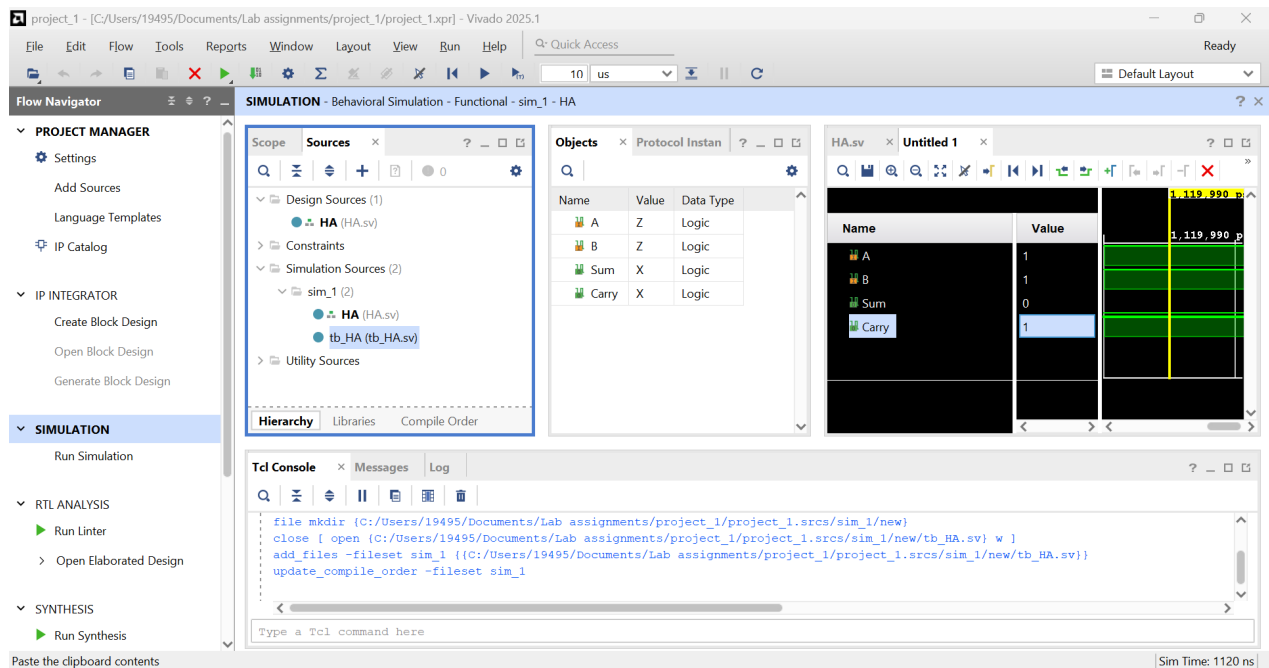


In the Add Sources window, select Create File to make a new simulation file. Enter a name for your testbench; in this example, we will call it tb_HA (short for “testbench for Half Adder”). After naming the file, click OK and then Finish.

Tip: It is good practice to use a clear and consistent naming convention for testbenches (for example, prefixing them with tb_). This makes it easy to distinguish between design files and simulation files in larger projects.



Now in the source window under Simulation sources, you can see your testbench file.



Double click on the tb_HA and use the code below.

Code 2: Testbench for Half Adder.

```
module tb_HA;

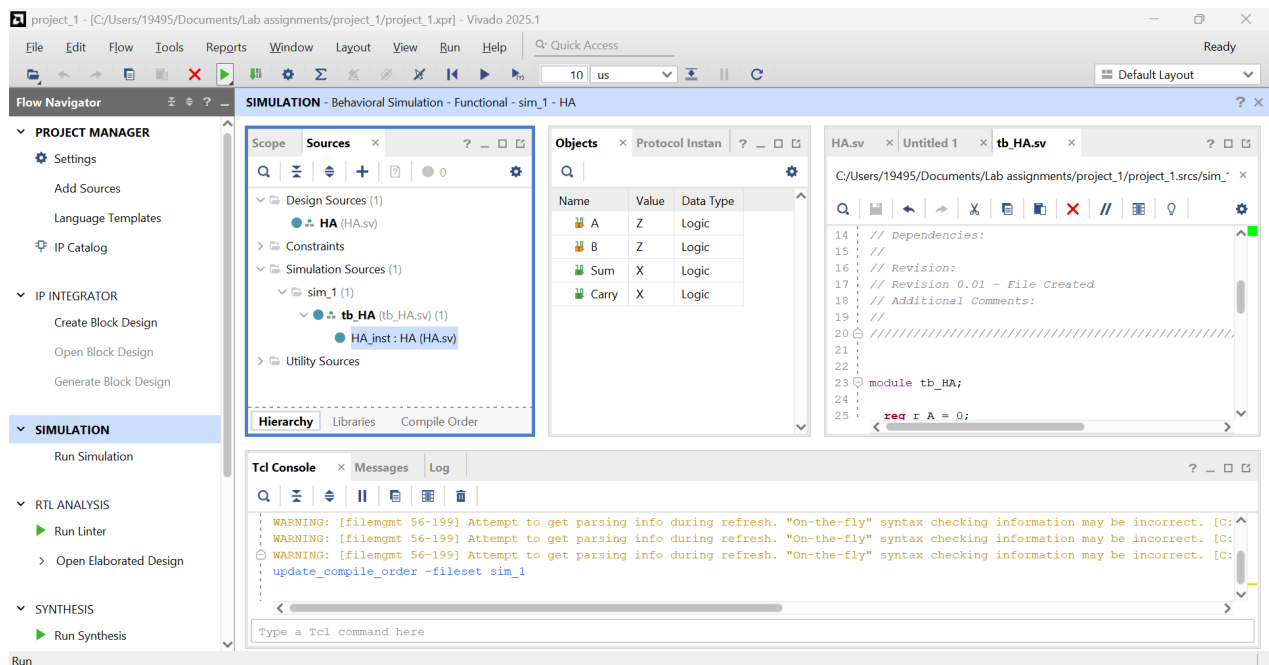
    reg r_A = 0;
    reg r_B = 0;
    wire w_Sum;
    wire w_Carry;

    HA HA_inst
    (
        .A(r_A),
        .B(r_B),
        .Sum(w_Sum),
        .Carry(w_Carry)
    );

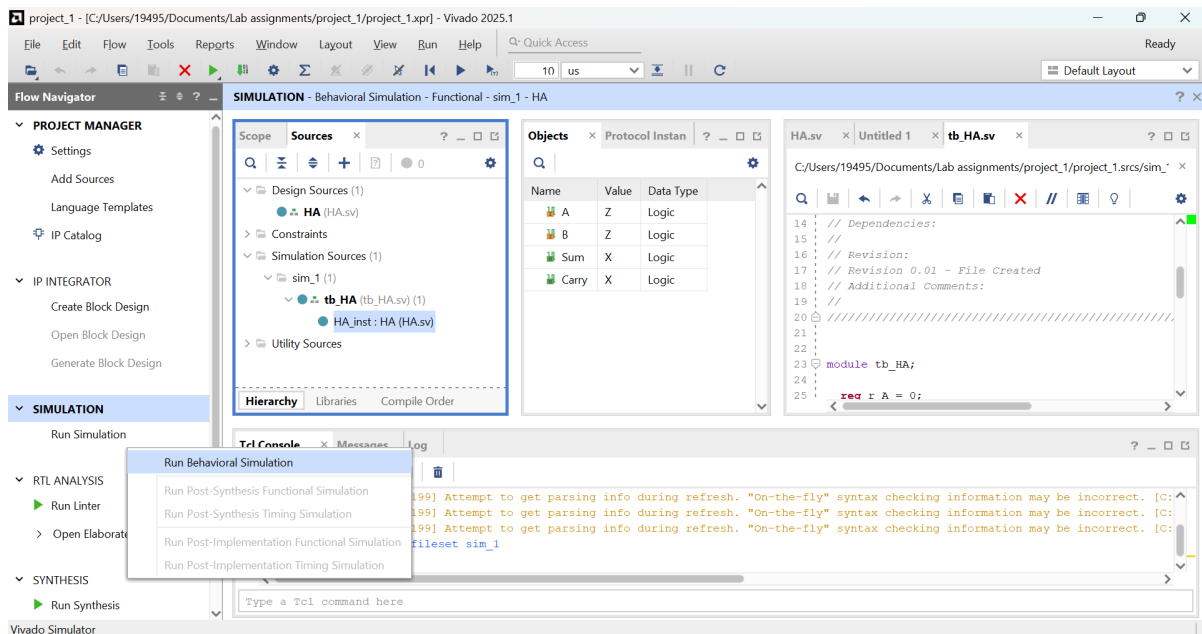
    initial
    begin
        r_A = 1'b0;
        r_B = 1'b0;
        #10;
        r_A = 1'b0;
        r_B = 1'b1;
        #10;
        r_A = 1'b1;
        r_B = 1'b0;
        #10;
        r_A = 1'b1;
        r_B = 1'b1;
        #10;
    end

endmodule // tb_HA
```

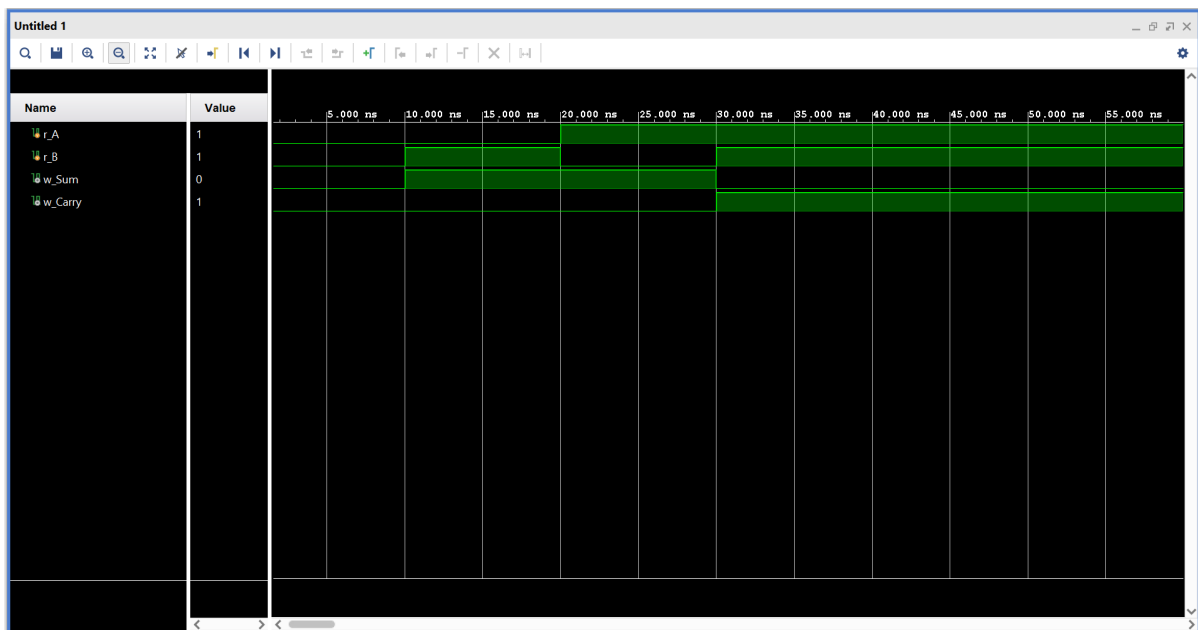
Save your code. Notice that your source window has been updated. Under Simulation Sources, the design module HA now appears as a submodule of tb_HA. If there are any syntax errors at this stage, they will be underlined in red. Be sure to correct all errors before continuing.



Now that the testbench is ready, run it by selecting Run Simulation from the Flow Navigator window.



You can zoom in on the waveform to examine the test results in detail and verify whether the output values match the expected behavior.



In this simulation, the inputs are tested across four time intervals:

- 0–10 ns: A = 0, B = 0 → expected outputs: Sum = 0, Carry = 0

- 10–20 ns: $A = 0$, $B = 1 \rightarrow$ expected outputs: Sum = 1, Carry = 0
- 20–30 ns: $A = 1$, $B = 0 \rightarrow$ expected outputs: Sum = 1, Carry = 0
- 30 ns onward: $A = 1$, $B = 1 \rightarrow$ expected outputs: Sum = 0, Carry = 1

By comparing the waveform outputs with the truth table of the Half Adder, you can confirm that your design is working correctly.